CS-2008: Numerical Computing

Semester project

This project must be completed under the following rules and guidelines:

Team Formation

- This is officially a **2-member project**.
- **Individual (solo) submissions are also allowed.**
- All students **must write their group members' names in the official group sheet**.
    - **If a group does NOT fill the sheet, they will NOT be assigned a presentation slot.**
    - No slot → **No presentation marks**.

Plagiarism Policy

- Plagiarism of any kind (reports, code, text, results) is strictly forbidden.
- Copying from classmates, seniors, GitHub, or the internet without proper citation will result in:

    - **Zero marks** for all involved students.
    - Possible academic penalties under university policy.

Submission Deadline

- **7 December 2025** (no extensions unless officially announced).
- Late submissions may incur mark deductions or rejection.

Presentation Instructions

- Presentations will take place in the **assigned slots only**.
- Slot allocation will be made **after the group sheet is finalized**.
- Students who miss their allocated slot will receive **0 marks** for the presentation.
- Presentation duration: **10–12 minutes per group**.
- Slide submission is optional, but **presenting is mandatory**.

Required Deliverables

You must submit all four items:

1. **Full Project Report**
2. **Summary Report (2–3 pages)**
3. **Complete and documented Source Code**
4. **Presentation (delivery mandatory)**

Failure to submit any component may reduce marks.

Optimization of HPCG by Improving SpMV or SymGS Performance

1. Introduction

High Performance Conjugate Gradient (HPCG) is a modern benchmark designed to measure the *real-world* performance of computing systems. Unlike the High-Performance Linpack (HPL) benchmark—which focuses on dense linear algebra—HPCG stresses operations commonly found in scientific applications such as:

- **Sparse matrix operations**
- **Memory bandwidth and latency**
- **Multigrid components**
- **Global communication overhead**

Your semester project focuses on studying the internal workflow of HPCG and improving the performance of **one** of its two major kernels:

1. **Sparse Matrix–Vector Multiplication (SpMV)**
2. **Symmetric Gauss–Seidel Smoother (SymGS)**

Each team will select **one** kernel, analyze a research paper, implement the optimization strategy, and integrate your improvement into the HPCG reference implementation.

2. Background (Provided for Understanding)

You do *not* need to research these topics on your own; this section builds context.

2.1 Why HPCG?

HPCG is built to stress parts of the system that real applications depend on:

- Memory speed & latency
- Sparse computations
- Irregular access patterns
- Multilevel communication (multigrid V-cycle)

Major components of HPCG include:

- SpMV
- SymGS smoother
- Restriction & prolongation operators
- Vector updates
- Dot products
- Multigrid V-cycle

2.2 Sparse Matrix Formats (Quick Overview)

Sparse matrices store only nonzero values. Common formats:

- **COO** – triplets (row, column, value)
- **CSR** – row pointers + column indices + values
- **ELL (ELLPACK)** – equal-length rows with padding
- **DIA** – stores diagonal values
- **SELL / SELL-P** – sliced versions of ELL for better cache behavior

Suggested reference:
A Systematic Literature Survey of Sparse Matrix-Vector Multiplication.

3. Project Options (Choose One)

Option A: Optimize SpMV

Use one of the provided papers, e.g.:

- **“CVR: Efficient Vectorization of SpMV on x86 Processors”**

Your tasks:

- Implement the data format proposed in the paper **OR** design your own inspired format
- Explain why your approach improves performance
- Integrate into HPCG and measure kernel and overall performance

Option B: Optimize SymGS

Use the attached documents/papers such as:

- *Optimizing Multi-grid Computation and Parallelization on Multi-cores*
- *SymGS Optimization Idea Notes*

Your tasks:

- Rewrite the optimization idea in your own words
- Implement the improved SymGS routine
- Show performance improvements with evidence
- You may propose your own improvement

4. Project Deliverables

You must submit:

4.1 Full Project Report

Must include:

1. Selected kernel (SpMV or SymGS)
2. Selected research paper
3. Short summary of the paper
4. Why you selected this paper

5. Critical review: What is good? What is weak?
6. Your proposed optimization (with diagrams/figures)
7. Implementation details
8. Architectural description of your system
    - Number of sockets
    - Cores / threads
    - Base & boost CPU frequency
    - L1 / L2 / L3 cache
    - RAM type and speed
9. Experimental setup
10. Performance evaluation:
    - Original HPCG kernel vs. your optimized version
    - Explain *why* performance changed

4.2 Summary Report (2–3 pages)

A shorter version of the main report.

Critical review, your choice of work, architecture used, analysis

4.3 Source Code

All code must be clean, readable, and runnable.

4.4 Presentation (10–12 minutes)

Slides submission is optional, but *presenting is mandatory*.

5.HPCG Reference Implementation

You must:

- Clone the official HPCG repository
- Build it on your system
- Run the baseline version
- Replace the selected kernel with your optimized implementation
- Verify correctness

5.1 Official Links

- Website: https://www.hpcg-benchmark.org
- Source Code: https://github.com/hpcg-benchmark/hpcg

5.2 HPCG Installation (General Steps)

1. Clone the repository
2. Use CMake to configure
3. Build using Make
4. Run HPCG with a specified problem size (default or custom)

6. Project Requirements Summary

- You must optimize **either SpMV or SymGS**
- Even if your total HPCG score decreases, your **kernel-level performance must improve**
- You must include system architecture details
- Additional optimizations = bonus points

7. Optional Interaction

A Q&A session will be arranged to help students understand:

- HPCG internals
- SpMV/SymGS optimization approaches

8. Final Notes

This project requires:

- creativity
- understanding of architecture
- coding skills
- benchmarking
- performance interpretation

Your evaluation will be based on:

- Correctness and clarity
- innovation
- depth of analysis

Good luck!